

Applied Machine Learning - Advanced

Prof. Daniele Bonacorsi, Dott. Luca Clissa

Lecture 1

[16.04.2026]

Data Science and Computation PhD + Master in Bioinformatics
University of Bologna

Applied ML “basic”: course recap

Recap [1/3]

Definition of ML

Types of ML

Supervised ML (vs Unsupervised)

Univariate linear regression

- cost function, GD

Multivariate Linear regression

- cost function, GD
 - ❖ feature scaling, learning rate, polynomial regression

Model representation

- computing parameters analytically with the normal equation

Recap [2/3]

Logistic regression

- classification
 - ❖ hypothesis representation, decision boundaries
- logistic regression model
 - ❖ cost function
 - ❖ simplified cost function and GD
 - ❖ advanced optimisations
 - ❖ multiclass classification

Regularization

- the problem of overfitting
- cost function
- regularised linear regression
- regularised logistic regression

Recap [3/3]

Advices for applying ML

- evaluating a learning model
 - ❖ evaluating a hypothesis
 - ❖ model selection and TVT sets
- Bias and variance
 - ❖ Diagnosis
 - ❖ Regularisation and bias/variance
 - ❖ Learning curves
- Error metrics for skewed classes
 - ❖ trade-off precision vs recall
 - ❖ ROC and AUC

Hands-on 100% with pure **sklearn**

From Bas to Adv

Material and tools for the Basic and Advanced part

You need to install NOTHING.

- You just need a **network** and a **browser**

The course is definitely **fluid** over time.. Changes are **needed**, always, to stay up to date.. but the discipline moves very **fast**, so changes are **many**.. Effort to keep the course up-to-date but at the same not to disrupt it completely in one AY

- part 1 used mostly sklearn
- part 2 uses mostly Keras+TensorFlow

You can code in the lectures slots (less and less..) and/or at home (more and more..)

- I am giving you paths and guidance code, and you explore at your pace

DISCLAIMER before starting

I am assuming that I give that AML-Basic content from granted

- Either you attended the BASIC part, or you did not because you knew everything already!

So:

- we will give for granted we know how to work on Colab..
- we will use python with less mercy..
- we will use numpy, pandas, matplotlib, seaborn, etc..
- we will use sklearn with a bit of familiarity and less guidance..

We will focus on what needs to be added up on top of all this!

In case of troubles/difficulties to follow, speak-up or **contact us**

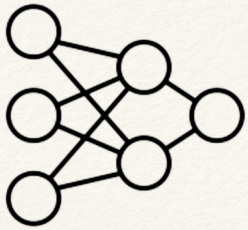
Jump from Basic to Advanced

We will jump.

From not-NN algos, to just NNs deep and forever. And back.

IMO it is the (educationally) right way to go

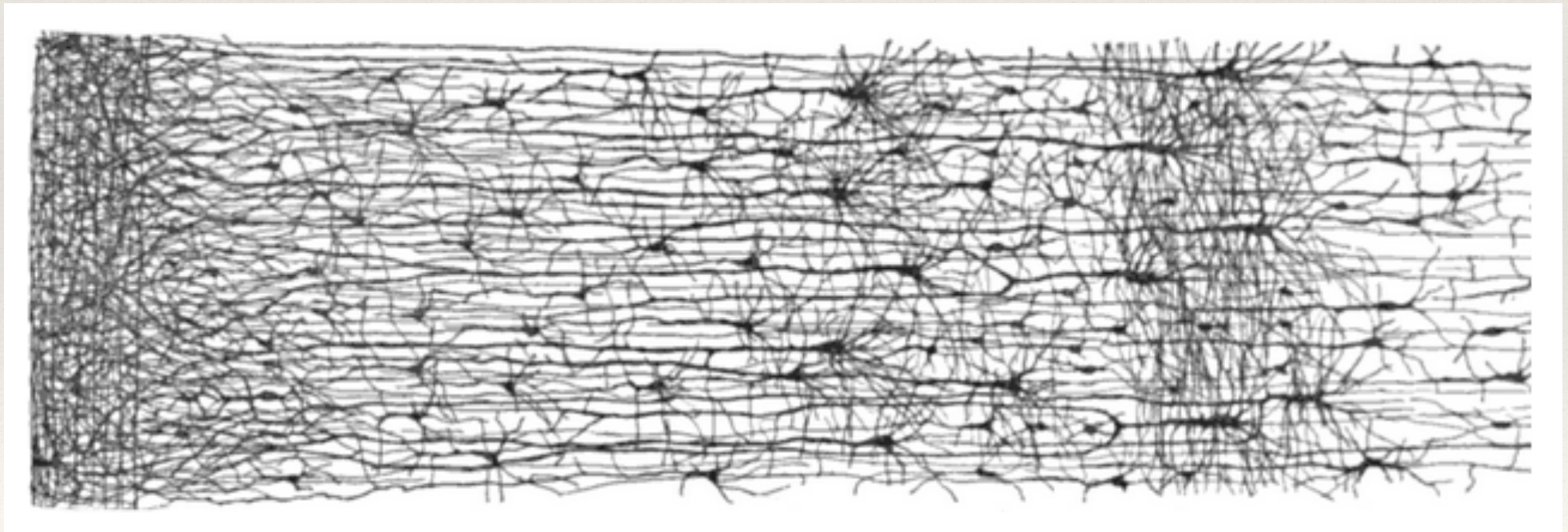
- Anyway, you have a FINAL QUESTIONNAIRE to state your feedback :)



NNs - intro

BNN

The architecture of biological neural networks (BNN) is still the subject of active research, but some parts of the brain have been mapped, and we know that **neurons are relatively simple** and they are **connected in large networks**, that are **often organized in consecutive layers...**

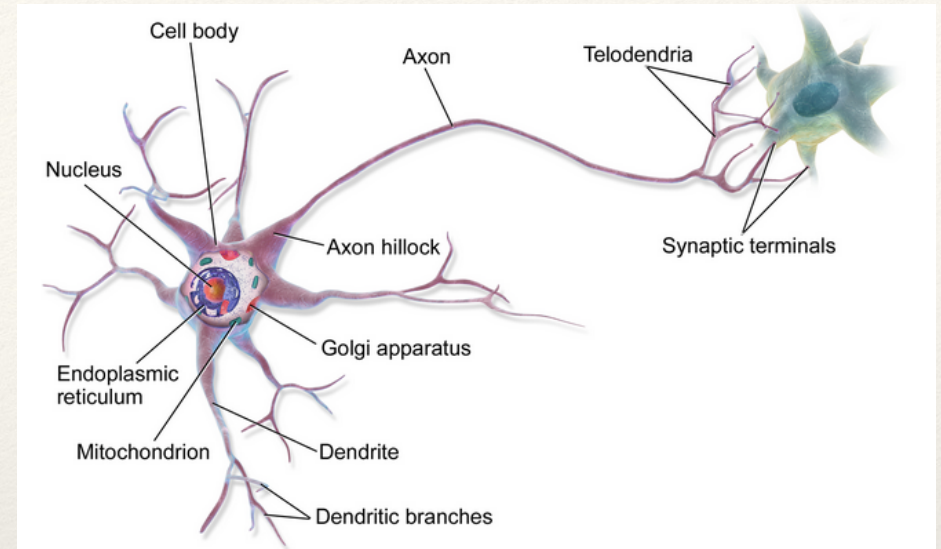


From BNN to ANN

ANN as simulation of Biological NN

- NN are developed by simulating networks of human neurons

The human neuron has a cell body, a number of input wires (*dendrites*) and an output wire (*axon*)



- we can think of it as a **computational unit** that gets inputs, computes, and spits output to other neurons (which we will call **nodes** or **units**)
- we can think of it simply as a **logistic unit** that computes some h as a sigmoid

Communication in BNNs is done via pulses of electricity

- we can think of **I/O** passing in/out **numerical values**
- a **wide** network..

Vastly oversimplified model..

Why NNs as ML algorithms?

Important to model **non-linear relationships among attributes**.

NNs are at the very core of **Deep Learning**

- versatile, powerful, scalable

Ideal to tackle large and highly complex ML tasks, such as:

- classifying billions of **images** (e.g. Google Images)
- powering **speech recognition** services (e.g. Siri, Alexa, ..)
- **recommending** the best videos to watch to $O(100M)$ users every day (e.g. YouTube)
- **learning to beat humans** in very specific tasks (e.g. from medical applications - e.g. radiology - to gaming - e.g. Go and DeepMind's AlphaZero)

Brief history of NNs

1943 (!): first introduction by McCulloch and Pitts

- a neurophysiologist and a mathematician
- *"A Logical Calculus of Ideas Immanent in Nervous Activity"*
- the first ANN architecture in human history
 - ❖ a simplified computational model of how biological neurons might work together in animal brains to perform complex computations using propositional logic

1960s: the successes of ANNs stopped - the **"first AI winter"**

Early 1980s: revival of interest in connectionism, new architectures, better training techniques. Slow progress, though.

1990s: other (not NN) powerful ML techniques were invented, e.g. SVMs. Seemed to offer better results and stronger theoretical foundations than ANNs - the **"second AI winter"**

2000s-2010s: a new "AI spring": **why now? will it last?**

Acceleration in 2010s on ANNs

[DISCLAIMER: also personal opinions here]

Opinions may vary, but IMO main actors have been:

- the raise of **Big Data** to train ANNs
 - ❖ often outperform other ML techniques on very large and complex problems
- the **technology progresses** (e.g. GPUs, TPUs, FPGAs, ..)
 - ❖ tremendous increase in computing technology since the 1990s, training time down by large factors, in part due to Moore's Law, in part to gaming industry (powerful GPU cards)
- "Democratisation" [*] of massive computing resources via **Cloud** approaches
 - ❖ [*] not free-of-charge, based on digital companies' business models, debatably "democratic" in a social sense, but offer options unavailable with on-premise facilities
- training algos improvements from the 1900s
 - ❖ fairly? minor.. but relatively small tweaks had a huge positive impact, still

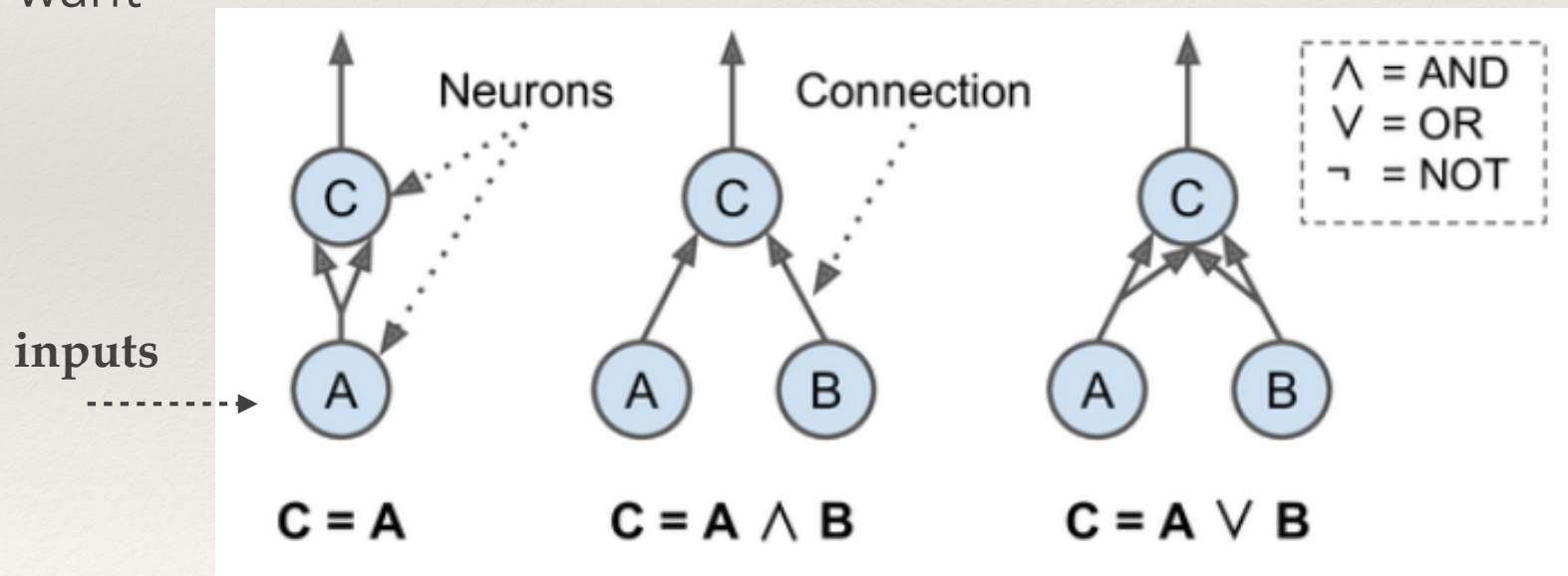
Today, it is a fact that **ML/DL is among the core transformative technologies at the basis of most world-wide activities aiming at extracting actionable insight from (big) data**

Logical computation with neurons

The McCulloch/Pitts model for a biological neuron was later called “**artificial neuron**”

- it has 1 (or more) **binary (on/off) inputs** and 1 **binary** output (*NOTE: all binaries!*)
- it activates its output when more than a certain number of its inputs are active (e.g. 2 in the examples below)

They showed that even with such a simplified model it is possible to build a network of artificial neurons that computes **any basic logical proposition** you want

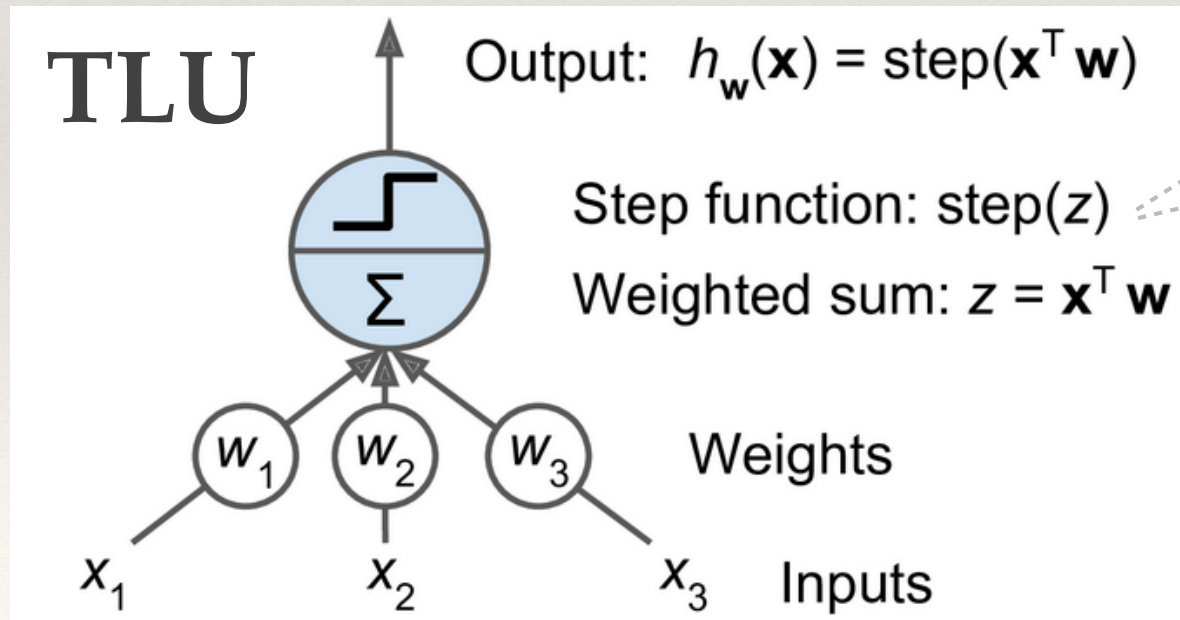


Imagine yourself how to combine them to form more complex logical functions

The TLU and the Perceptron

1957, Rosenblatt: "Perceptron" as one of the simplest possible ANN architectures, based on a slightly modified artificial neuron, called **threshold logic unit (TLU)** or linear threshold unit (LTU)

- the **inputs/output** are now **numbers** (not binary on/off values)
- each input connection is associated with a **weight**
- the TLU computes a **weighted sum of its inputs**, applies a **step function** to that sum, and **outputs** the result



$$\text{heaviside}(z) = \begin{cases} 0 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{cases}$$

$$\text{sgn}(z) = \begin{cases} -1 & \text{if } z < 0 \\ 0 & \text{if } z = 0 \\ +1 & \text{if } z > 0 \end{cases}$$

1 single TLU can act as a simple linear binary classifier

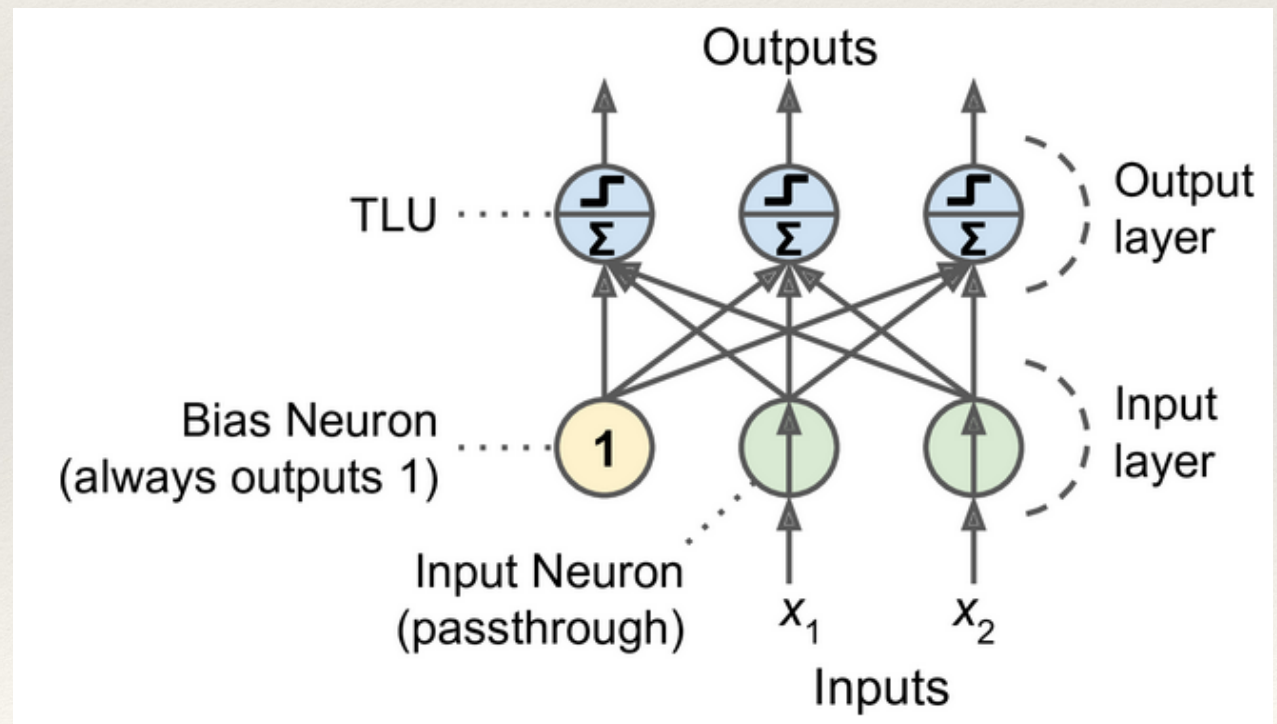
- linear combination of the inputs, and depending of the thresholds → positive/negative class (just like Logistic Regression classifier or a linear SVM !)

The TLU and the Perceptron

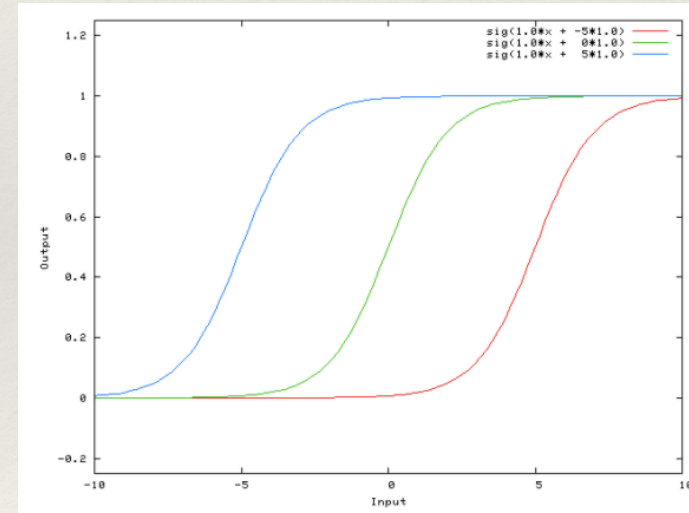
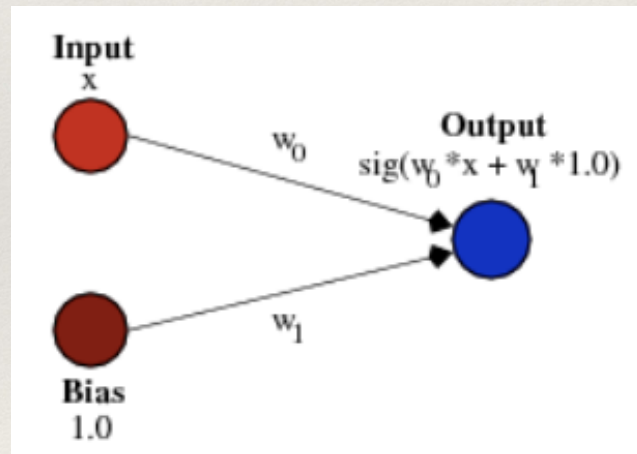
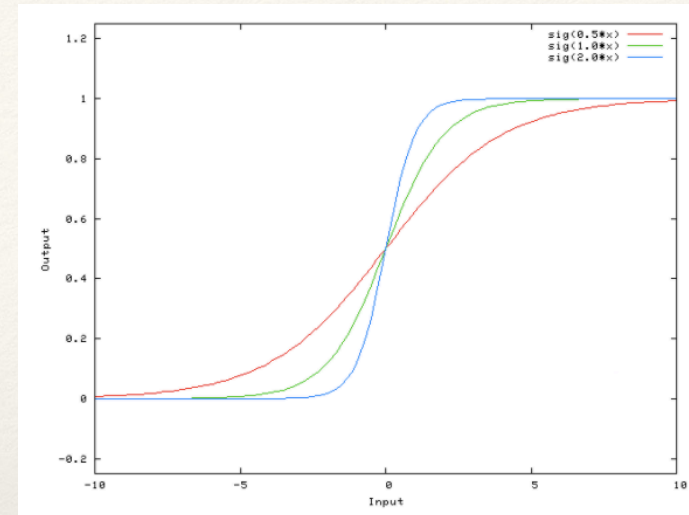
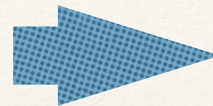
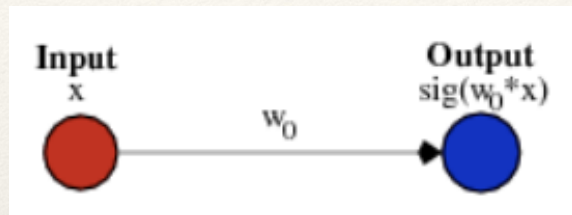
A **Perceptron** is then composed of a **single layer of TLUs** with each TLU connected to all the inputs

- in the **input layer**, an extra bias feature is generally added ($x_0 = 1$), via a “**bias neuron**” that just outputs 1 all the time.
- When all the neurons in a layer are connected to all the neuron in the previous layer (i.e. here the input neurons), it is called a **fully-connected** or **dense** layer

This Perceptron can classify instances simultaneously into 3 different binary classes, which makes it a linear multi-class classifier



Why a bias neuron: the long answer



A bias value allows you to shift the activation function to the left or right, which may be critical for successful learning.

Why a bias neuron: the short answer

Why?

Simple analogy: just like b in $y=ax+b$.

Training a Perceptron

Input from biology and neurosciences

- 1949, Hebb; *"Organization of Behavior"* [\[Ref-Hebb\]](#)
- **Hebb's rule** (Hebbian learning): when a biological neuron triggers another neuron, the connection between these two neurons **grows stronger**
 - ❖ Lowel: **"Cells that fire together, wire together"**

Largely inspired by Hebb's rule, the Perceptron training algorithm proposed by Rosenblatt was a **weight update**

- the Perceptron is fed one training instance at a time. For each instance it makes its prediction(s). For every output neuron that produced a wrong prediction, it reinforces the connection weights from the inputs that would have contributed to the correct prediction

$$w_{ij}^{(\max)} = w_{ij} + \eta (y_j - \hat{y}_j) x_i$$

$w_{i,j}$ is the connection weight between the i^{th} input neuron and the j^{th} output neuron.

x_i is the i^{th} input value of the current training instance.

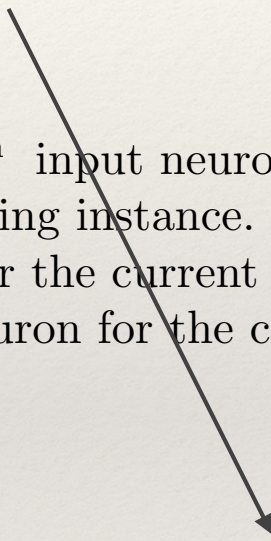
$\hat{y}_j$ is the output of the j^{th} output neuron for the current training instance.

y_j is the target output of the j^{th} output neuron for the current training instance.

η is the learning rate.

$$w_{ij}^{(\max)} = w_{ij} + \eta (y_j - \hat{y}_j) x_i$$

$w_{i,j}$ is the connection weight between the i^{th} input neuron and the j^{th} output neuron.
 x_i is the i^{th} input value of the current training instance.
 $\hat{y}_j$ is the output of the j^{th} output neuron for the current training instance.
 y_j is the target output of the j^{th} output neuron for the current training instance.
 η is the learning rate.

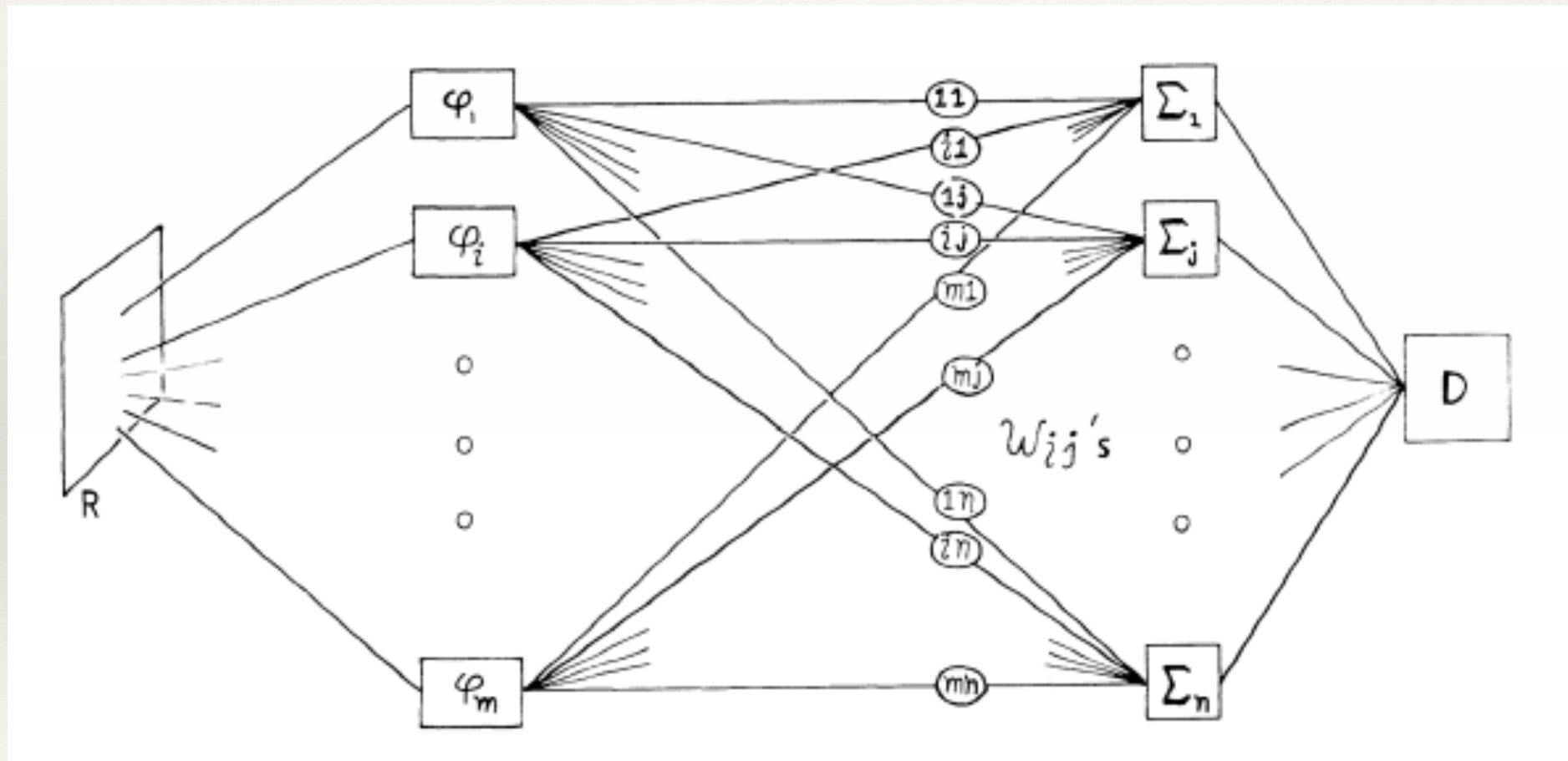


The decision boundary of each output neuron is **linear**..
so Perceptrons are incapable of learning complex patterns...

Note: you can implement a Perceptron in sklearn, and compare it to LogisticRegression.
Works like charm - but it does not output class probabilities. Nice idea, but weak.. yet..

Multi-Layer Perceptron (MLP)

1969, Minsky/Papert, "Perceptrons", the monograph [\[Ref-MinskyPapert\]](#)



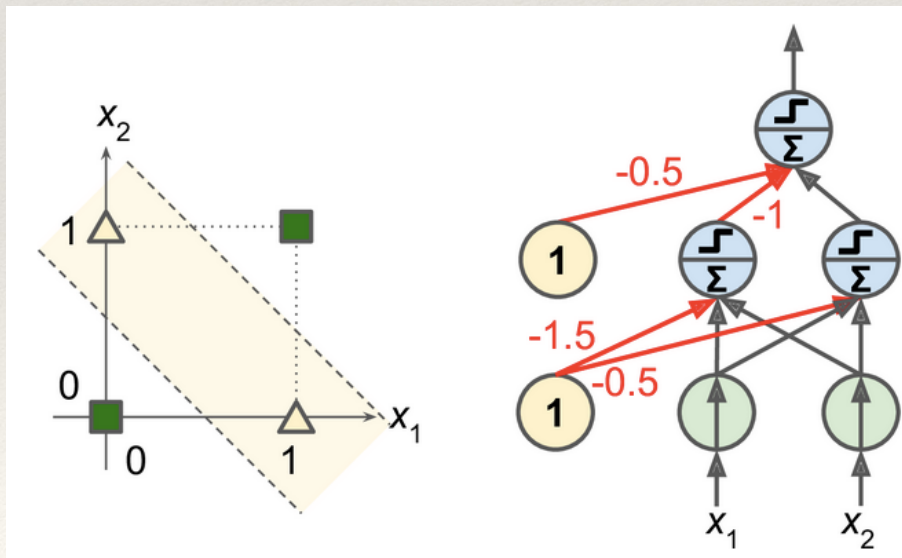
Multi-Layer Perceptron (MLP)

Minsky/Papert:

- serious weaknesses of Perceptrons highlighted, in particular its incapability of solving relatively trivial problems, e.g. the exclusive-OR (XOR) classification problem
- Well, true of any other linear classification model (e.g. Logistic Regression classifiers).. but **researchers had expected much more from Perceptrons!** Great disappointment. People dropping off the field..

But.. it soon turned out most limitations of Perceptrons could be eliminated by **stacking multiple Perceptrons!** → first concept of "**layers**"!

- the resulting ANN is called a **Multi-Layer Perceptron (MLP)**



Note that **you added an entire brand new layer** to the original Perceptron architecture

(see next)

MLP - FCNN - FFNN

Signal flows only in one direction:
Feed Forward NN (FFNN)

layers of TLUs

Output layer

Hidden layer

Activation functions important to have some non-linearity between layers

Input layer

all layers are FC

deep stack of hidden layers?

→ **Deep NN (DNN)**

(Deep Learning sometimes used also for shallow, non deep, NN)

A bias unit for each layer apart from the output one

